

SmartScheduler++: Intelligent CPU Scheduling in Linux CFS Using eBPF and Machine Learning

Pankaj Suman, Smita Das*

Department of Computer Science & Engineering
NIT Agartala,
Tripura - 799046 , INDIA
email:smitadas.nita@gmail.com

Abstract. This paper introduces SmartScheduler++, an adaptive CPU scheduling framework for the Linux Completely Fair Scheduler (CFS) that integrates eBPF based observability and machine learning to enhance CPU scheduling. The framework uses low level scheduler trace points to collect runtime data such as execution time, wait time, wakeups, and context switches with low overhead. The data is then converted into fifteen features that represent system efficiency, responsiveness, and fairness. The proposed solution is tested using various classification algorithms, including Random Forest, Gradient Boosting, Logistic Regression, and k Nearest Neighbors, with varying observation windows. The experimental evaluation shows that ensemble classifiers perform better than linear and distance based classifiers, with Random Forest and Gradient Boosting achieving the best accuracy and F1 scores. The evaluation also shows that longer observation windows lead to better performance, achieving optimal accuracy of 97% at 45 seconds with Random Forest.

Keywords: Completely Fair Scheduler, extended Berkeley Packet Filter, CPU Scheduling, Linux kernel System.

1 INTRODUCTION

CPU scheduling is an elementary feature of the present day operating system which strongly impact [1] machine productivity and throughput. In contrast, Linux, which is extensively utilized in server and cloud infrastructure, follows an analogy named Completely Fair Scheduler (CFS) [2], which assigns a processor span between executing programs. Nevertheless, the intent of scheduler ensures fairness, although previous theories have found inability such as load imbalance, delayed task wakeup, notably on multi-core systems [3]. Recent advances in extended Berkeley Packet Filter (eBPF) allows secure, low overhead, real time monitoring of kernel scheduling events through bpftrace, perf and trace points commands.

In real world scenarios, the Linux CFS often exhibits suboptimal behavior under production workloads. Prior studies [4] indicate that CFS may leave CPU cores idle while runnable programs remain waiting. Moreover, CFS makes scheduling decisions using heuristics, but its predictive power is limited. As a result, it often struggles to deliver consistent throughput in database servers and applications that require heavy

synchronization [5]. Because CFS relies so much on past execution history, it struggles to keep up with workloads such as: microservices, mixed I/O–CPU applications, and cloud native systems [6]. Real-world observations using eBPF show that this leads to delayed scheduling decisions and fairness issues during runtime [7]. These problems increase tail latency and reduce overall system productivity. To address these challenges, this work introduces SmartScheduler++, an eBPF driven, machine learning based adaptive CPU scheduler for Linux CFS. The main contributions of this paper are:

- A lightweight eBPF tracing framework is designed to capture fine grained, real time Linux scheduler metrics (runtime, waiting time, wakeups, context switches, etc.) without requiring kernel recompilation.
- CPU scheduling is reformulated as a learning problem, where machine learning models characterize task behavior and enable workload aware scheduling decisions that dynamically adapt to both CPU bound and I/O-bound workloads.

The rest of the paper is structured as follows: Section 2 discusses recent literature along with background, Section 3 depicts Proposed Methodology in details followed by Result & Analysis in Section 4. Finally, the paper is concluded with future work in Section 5.

2 RELATED WORK

CFS improves multitasking by assigning fractional time slices based on task weights derived from nice values. Instead of fixed time slices, CFS organizes execution using virtual runtime (vruntime) and selects the task with the smallest vruntime from a red-black tree, achieving efficient scheduling with $O(\log n)$ complexity [8].

Despite its design for long-term fairness, CFS has been shown to suffer from load imbalance [9], delayed wakeups, upward task migration, and idle cores in multicore environments. Prior studies further demonstrate that machine learning (ML) techniques can predict task behavior [4], offering opportunities to enhance scheduling efficiency. [10] demonstrated that the Completely Fair Scheduler (CFS) can violate the principle of work conservation, resulting in significant under utilization of CPU resources in real world applications such as databases and parallel build systems. Comparative studies between Linux CFS and Windows schedulers further reveal that while CFS generally achieves lower latency for interactive tasks, its performance degrades under heavy background load and mixed workload scenarios [2].

To overcome these limitations, researchers have explored machine learning based scheduling enhancements such as predicting CPU burst times using historical execution behavior to reduce turnaround time and context switch overhead [11]. Subsequent work applied supervised learning techniques, including tree based models, to dynamically adjust priorities and improve scheduling efficiency [12]. However, most of these approaches depend on offline training or require invasive kernel modifications [13], which restricts their applicability in production systems. Recent research has shifted focus toward improving scheduler observability rather than direct kernel modification. The advent of the eBPF has enabled safe, lowoverhead, and non intrusive tracing of kernel scheduling events such as runtime, waiting time, wakeups, and context switches

[7]. eBPF-based studies have been used to analyze fairness properties of CFS [14] and to identify runtime scheduling anomalies in production environments. In parallel, learning-based schedulers leveraging deep learning and reinforcement learning have been extensively studied in cloud, fog, and multi-cloud environments. These approaches optimize metrics such as makespan, latency, and energy consumption, but they operate above the operating system kernel and do not directly influence Linux CPU scheduling decisions [15].

3 PROPOSED METHODOLOGY

In this section, we describe the system architecture along with dataset description.

3.1 System model

The proposed model is depicted in Figure 1. The system model contains the smartschedu-

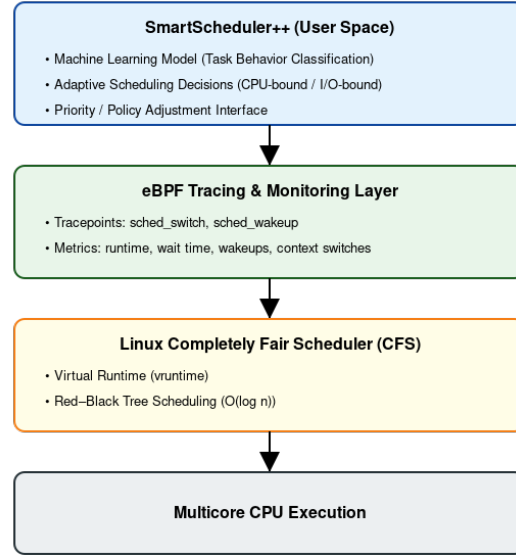


Fig. 1. Workflow Architecture of SmartScheduler++

lar++ user space, eBPF Tracing & Monitoring Layer, Linux Completely Fair Scheduler and Multicore CPU execution.

SmartScheduler++ functions as the intelligent decision-making layer in user space, interacting with the kernel via eBPF hooks. An ML classifier evaluates process behavior using metrics such as CPU usage, wait times, and I/O activity. Tasks are categorized into

CPU-bound, I/O bound, or latency sensitive types using models like Random Forest, Gradient Boosting, Logistic Regression and K-Nearest Neighbor. Based on classification, SmartScheduler++ dynamically adjusts task priorities for real time responsiveness. These decisions evolve continuously with runtime behavior.

eBPF Tracing & Monitoring Layer is a subsystem that bridges SmartScheduler++ with the Linux scheduler. By attaching eBPF programs to key trace points, it captures context switches, wakeups, and latency events with near zero overhead. It computes real time metrics such as: runtime, wait time, wake up frequency and context switches. The following commands list trace points, inspect their fields, and ensure events are not lost:

```
bpfftrace -l 'tracepoint:sched:*' (1)
```

```
bpfftrace -lv tracepoint:sched:*' (2)
```

```
BPFTRACE_PERF_RB_PAGES = 8192 (3)
```

Linux Completely Fair Scheduler (CFS) is the default CPU scheduler, ensuring fairness by allocating time based on virtual runtime. Tasks with lower vruntime are prioritized, preventing starvation. SmartScheduler++ influences CFS indirectly via priority and policy adjustments.

Multicore CPU Execution integrates multiple cores on a single chip, enabling parallel instruction processing and higher throughput. Each core runs threads independently with private caches, coherence protocols, and pipelining. SMT, branch prediction, and interconnects further enhance scalability, efficiency, and performance.

3.2 Dataset description

This work utilizes a scheduler centric dataset derived from Linux kernel trace-points using eBPF instrumentation and can be found at ¹. Raw scheduler events have been collected from a Linux system using the bpfftrace tool ². Four stable Linux scheduler trace points are used as follows: sched_stat_runtime, sched_stat_wait, sched_wakeup, sched_switch. These trace points expose complementary aspects of scheduling behavior, including execution time, waiting latency, wakeup activity, and context switch frequency. Each record in the raw dataset contains the event type, high resolution timestamp (nanoseconds), process identifier (PID), process name, CPU core, and an event specific value and is depicted in Figure 2. From the four aggregated base metrics, additional derived features were computed to quantify execution efficiency, interactivity, scheduling overhead, and fairness. The final dataset contains fifteen features described in Table 1.

4 RESULTS & ANALYSIS

In this section, experiments have been carried out using various ML models such as: Random Forest, KNN, Logistic Regression and Gradient Boost.

¹ <https://github.com/princeypiexd/SMARTSCEDHULAR-.git>

² <https://github.com/bpfftrace/bpfftrace>

```
[prince@pi3d@pprocks ~]$ sudo sh -c 'echo "event,timestamp_ns,pid,comm,cpu,value"
> sched_raw.csv && \
BPFFTRACE_PERF_RB_PAGES=8192 timeout 30s bpftrace -e "
tracepoint:sched:sched_stat_runtime
{
    printf("runtime,%llu,%d,%s,%d,%d\n",
        nsecs, pid, comm, cpu, args->runtime);
}
tracepoint:sched:sched_stat_wait
{
    printf("wait,%llu,%d,%s,%d,%d\n",
        nsecs, pid, comm, cpu, args->delay);
}
tracepoint:sched:sched_wakeup
{
    printf("wakeup,%llu,%d,%s,%d,%d\n",
        nsecs, pid, comm, cpu);
}
tracepoint:sched:sched_switch
{
    printf("switch,%llu,%d,%s,%d,%d\n",
        nsecs, pid, comm, cpu);
}
" >> sched_raw.csv'
```

2

Fig. 2. bpftrace script capturing scheduler trace points

Table 1. Scheduler Feature Definitions

Feature	Definition	Insight	Feature Origin
total_runtime_ns	Total CPU execution time per window	CPU intensity	Primary
avg_runtime_ns	Mean runtime per scheduling event	Burst length	Derived
runtime_var_ns	Runtime variance (ns ²)	Burst variability	Derived
total_wait_ns	Total wait time in runqueue	Scheduling latency	Primary
avg_wait_ns	Mean wait time per event	Responsiveness	Derived
wait_var_ns	Wait time variance (ns ²)	Wait instability	Derived
wakeup_count	Number of wakeup events	Interactivity	Primary
switch_count	Number of context switches	Preemption rate	Primary
runtime_per_switch	Runtime normalized by switches	CPU efficiency	Derived
w_ratio	Wakeup per switch	Interrupt driven behavior	Derived
wait_runtime_ratio	Wait time to runtime ratio	Starvation pressure	Derived
cpu_util	Runtime normalized by window duration	CPU utilization	Derived
switch_rate	Context switches per second	Scheduler overhead	Derived
wakeup_rate	Wakeup per second	Task responsiveness	Derived
fairness_pressure	Composite wait/runtime metric	Scheduling fairness	Derived

Figure 3(a) shows the performance of the Random Forest classifier across different time windows. At 5 seconds, the model achieves high precision but low recall, leading to a reduced F1 score due to insufficient observation time. As the time window increases, recall and F1 score improve significantly while precision remains high. The best overall performance is achieved at 45 seconds, with the highest accuracy and F1 score. These results indicate that longer time windows (20–45 seconds) enable more accurate and balanced classification of CPU bound and I/O bound workloads. Figure

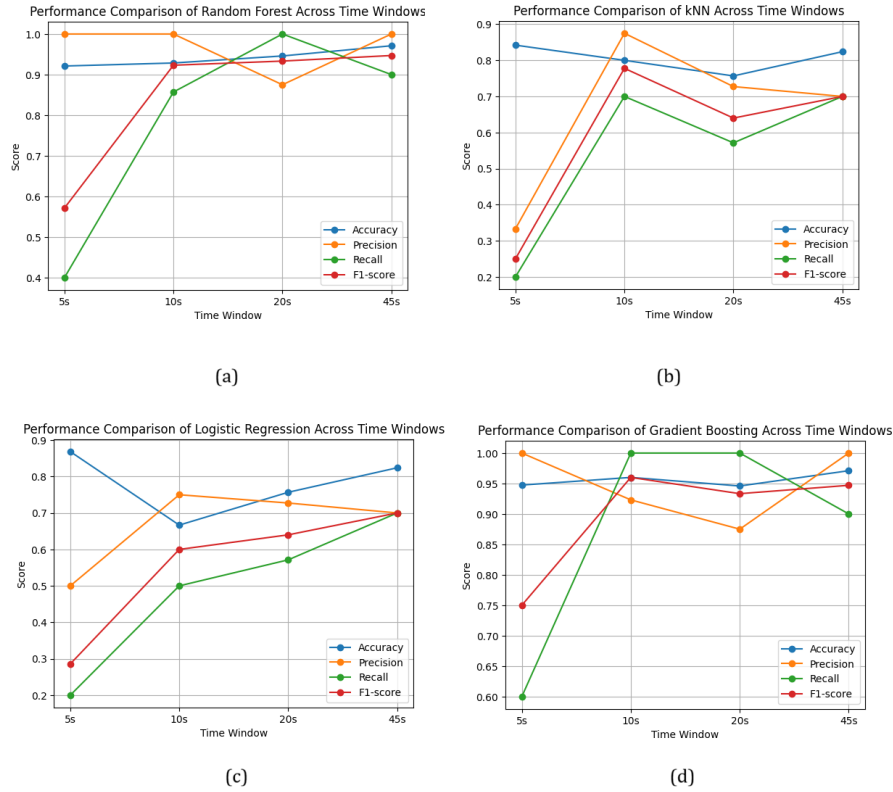


Fig. 3. Performance Metrics Analysis

3(b) shows kNN performance across varying time windows. At 5 seconds, recall and F1 score are low despite acceptable accuracy, reflecting poor CPU bound detection. Performance improves notably at 10 seconds, but degrades slightly at 20 seconds due to class imbalance. The best precision recall balance occurs at 45 seconds. Figure 3(c) presents the performance of the Logistic Regression classifier across varying time windows. At 5 seconds, accuracy is relatively high but recall and F1-score remain low, indicating weak detection of CPU-bound tasks. Performance improves steadily at 10 and 20 seconds, with the best precision & recall balance achieved at 45 seconds. Fig-

ure 3(d) illustrates Gradient Boosting performance across time windows. At 5 seconds, accuracy and precision are high but recall is lower, missing CPU bound tasks. At 10 and 20 seconds, recall peaks while accuracy and F1-score remain stable. The 45 second window delivers the best overall results, confirming Gradient Boosting’s robustness to window variations and suitability for smart scheduling.

Among the above mentioned ML algorithms its clear that RF depicts the best performance. Additionally, Figure 4 shows the performance improvement of Random Forest across various time windows. At 5 seconds, baseline accuracy is 0.92. Accuracy improves slightly at 10 seconds and gains further at 20 seconds with better class discrimination. The 45 second window achieves the highest accuracy (0.97) and most stable results, confirming that longer observation windows consistently enhance Random Forest accuracy for smart scheduling.

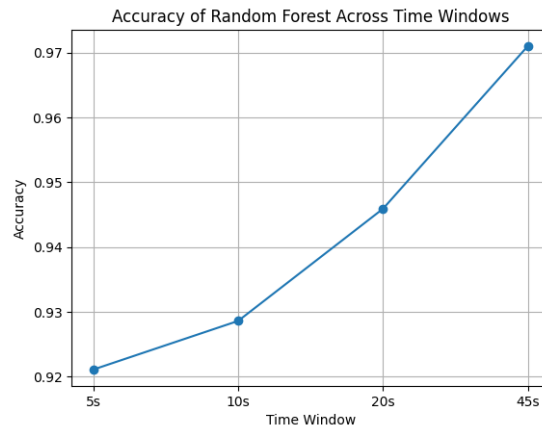


Fig.4. Improvement of Accuracy over Time

Accuracy measures the overall proportion of correctly classified workloads among all samples. While it provides a general performance overview, it may be misleading in the presence of class imbalance between CPU-bound and IO-bound tasks.

4.1 Comparative Analysis

Table 2 and Table 3 represents a comparative evaluation of Logistic Regression, kNN, Random Forest and Gradient Boosting classifiers across Short Time Window (5 - 10 seconds) and Long Time Window (20 - 45 seconds) respectively. Random Forest and Gradient Boosting demonstrate superior and stable performance across all time windows, with the best results observed at the 45 second window. This indicates the model’s capacity to capture complex scheduling patterns when sufficient observation time is available. In contrast, Logistic Regression and kNN exhibit lower and more variable performance, particularly at shorter time windows, due to their sensitivity to class im-

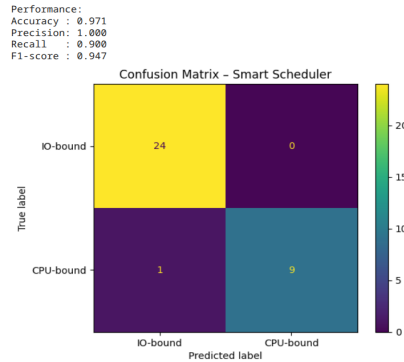


Fig. 5. Confusion matrix for the random forest algorithm

Table 2. Performance Comparison Across Classifiers for Short Time Windows

Model	Time	Accuracy	Precision	Recall
Random Forest	5 s	0.9211	1.0000	0.4000
	10 s	0.9286	1.0000	0.8571
Gradient Boosting	5 s	0.9474	1.0000	0.6000
	10 s	0.9600	0.9231	1.0000
Logistic Regression	5 s	0.8684	0.5000	0.2000
	10 s	0.6667	0.7500	0.5000
kNN	5 s	0.8421	0.3333	0.2000
	10 s	0.8000	0.8750	0.7000

balance and limited feature representation meanwhile fig 5 showcase the confusion matrix of random forest.

5 CONCLUSIONS & FUTURE WORK

This paper demonstrates the effectiveness of treating CPU scheduling as a learning problem by utilizing fine grained eBPF trace points to capture real time system behavior. The framework successfully translates raw scheduler events into meaningful features that reflect workload dynamics, enabling classifiers to adapt scheduling policies to diverse task types. Comparative evaluation highlights the strength of ensemble approaches in maintaining stability and accuracy across varying observation periods, while simpler models show sensitivity to imbalance and limited robustness. The future work involves integrating the ML driven decision logic into a user space daemon and exploring the possibility of eBPF based kernel space inference.

Table 3. Performance Comparison Across Classifiers for Long Time Windows

Model	Time	Accuracy	Precision	Recall	F1-score
Random Forest	20 s	0.9459	0.8750	1.0000	0.9333
	45 s	0.9710	1.0000	0.9000	0.9470
Gradient Boosting	20 s	0.9459	0.8750	1.0000	0.9333
	45 s	0.9710	1.0000	0.9000	0.9470
Logistic Regression	20 s	0.7568	0.7273	0.5714	0.6400
	45 s	0.8240	0.7000	0.7000	0.7000
kNN	20 s	0.7568	0.7273	0.5714	0.6400
	45 s	0.8240	0.7000	0.7000	0.7000

References

- Seelam, V., Yadav, S., Lahar, G.: A comparative analysis of cpu scheduling algorithms: Efficiency, fairness, and optimization in resource management. Fairness, and Optimization in Resource Management (November 14, 2024) (2024) [1](#)
- Wei-Cong Fan, C.S.W.: Comparison of interactivity performance of linux cfs and windows cpu schedulers. International Journal / IEEE Conference (2020) [1](#), [2](#)
- Goel, N.: Performance analysis of cpu scheduling algorithms with novel omdrrs algorithm. International Journal of Advanced Computer Science and Applications (2025) [1](#)
- Anusha G H . Minal K. B P Varsha, Dr. Geethishree Mishra, M.G.K.: Ml engine for linux completely fair scheduler. International Journal of Computer Systems (2022) [1](#), [2](#)
- Das, R., Singhal, R., Singh, R., Yadav, R.K.: Linux cpu schedulers: Cfs, muqss, bmj and cacule comparison. In: 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), IEEE (2023) 1–7 [2](#)
- Jesson Yo, A.I.K.: Analyzing fair share fairness of tasks in the linux completely fair scheduler using ebpf. IEEE Conference / Journal (2022) [2](#)
- Yakun Zhang, Han Wang, P.G.: Research on efficient packet filter technology based on ebpf. IEEE Access (2021) [2](#), [3](#)
- Harichane, I., Makhoul, S.A., Belalem, G.: Kubesc-rtp: Smart scheduler for kubernetes platform on cpu-gpu heterogeneous systems. Concurrency and Computation: Practice and Experience **34**(21) (2022) e7108 [2](#)
- Hayat, A., Khalid, Y.N., Rathore, M.S., Nadir, M.N.: A machine learning-based resource-efficient task scheduler for heterogeneous computer systems. The Journal of Supercomputing **79**(14) (2023) 15700–15728 [2](#)
- Lozi, J.P., Lepers, B., Funston, J., Gaud, F., Quéma, V., Fedorova, A.: The linux scheduler: A decade of wasted cores. In: Proceedings of EuroSys, ACM (2016) [2](#)
- Negi, A., P., K.K.: Applying machine learning techniques to improve linux process scheduling. In: Proceedings of IEEE TENCON, IEEE (2005) [2](#)
- Geng, T., Amaris, M., Zuckerman, S., Goldman, A., Gao, G.R., Gaudiot, J.L.: A profile-based ai-assisted dynamic scheduling approach for heterogeneous architectures. International Journal of Parallel Programming **50**(1) (2022) 115–151 [2](#)
- Naila Aslam¹, Nadeem Sarwar², A.B.: Designing a model for improving cpu scheduling. International Journal of Computer Science (2019) [2](#)
- Chen, J., Banerjee, S.S., Kalbarczyk, Z.T., Iyer, R.K.: Machine learning for load balancing in the linux kernel. In: Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '20), Tsukuba, Japan, ACM (August 2020) 1–8 [3](#)

15. Choppara, P., Mangalampalli, S.: An efficient deep reinforcement learning based task scheduler in cloud–fog environment. *Cluster Computing* **28**(1) (2025) 67–92 [3](#)